

forceTorqueThrForceMapping

1 Executive Summary

This module maps commanded forces and torques defined in the body frame of the spacecraft to a set of thrusters. It is capable of handling Center of Mass (CoM) offsets and non-controllable axes. In contrast to `thrForceMapping`, this module only handles on-pulsing, but not off-pulsing. Further, it provides a single force/torque projection onto the thrusters and thus lacks some of the robustness features of `thrForceMapping`.

The commanded force and torque input messages are optional, and the associated vectors are zeroed if no input message is connected. This provides a general capability to map control torques, forces, or torques and forces onto a set of thrusters.

2 Message Connection Descriptions

The following table lists all the module input and output messages. The module msg connection is set by the user from python. The msg type contains a link to the message structure definition, while the description provides information on what this message is used for. Both the `cmdTorqueInMsg` and `cmdForceInMsg` are optional.

Table 1: Module I/O Messages

Msg Variable Name	Msg Type	Description
<code>cmdTorqueInMsg</code>	<code>CmdTorqueBodyMsgPayload</code>	(optional) vehicle control torque ($\mathbf{L}_r$) input message
<code>cmdForceInMsg</code>	<code>CmdForceBodyMsgPayload</code>	(optional) vehicle control force input message
<code>thrConfigInMsg</code>	<code>THRArrayConfigMsgPayload</code>	thruster cluster configuration input message
<code>vehConfigInMsg</code>	<code>VehicleConfigMsgPayload</code>	vehicle configuration input message
<code>thrForceCmdOutMsg</code>	<code>THRArrayCmdForceMsgPayload</code>	thruster force command output message

3 Module Assumptions and Limitations

This module assumes that each thruster only produces positive thrust (on-pulsing only). The pseudo-inverse of the stacked force/torque mapping matrix $[D]$ is computed via a truncated SVD: singular values below the relative cutoff $\sigma_{\max} \cdot \varepsilon_{f32} \cdot \max(6, N_{\max})$ (the fp32 noise floor) are treated as zero, which projects uncontrollable directions in the 6-D command space out of the result. A thruster geometry whose kept singular subspace is ill-conditioned (condition number above 100) is rejected when the configuration is created, as the pseudo-inverse would otherwise amplify the command and fp32 round-off. The resulting solution is the minimum-norm least-squares projection onto the thruster set, shifted so the minimum thrust is zero. No off-pulsing, saturation handling, or thruster availability masking is performed within this module. All numeric computation uses single-precision (FP32).

4 Module Architecture

The module follows the three-layer adapter / algorithm / C-shim pattern shared across the fp32-fsw-xmera algorithms. A reader interested only in the math can skip to *Detailed Module Description* below; this section documents the messaging/lifecycle plumbing.

Adapter layer (`forceTorqueThrForceMapping.h/.cpp`) Inherits `SysModel`. Holds `desiredControlAxes_B` as a Phase-1 property exposed to Python (set before `reset()`). On `reset()`, the adapter validates input

messages, reads the thruster array (`thrConfigInMsg`) and center of mass (`vehConfigInMsg`), builds a validated `ForceTorqueThrForceMappingConfig` and constructs the algorithm via `std::make_unique`. Each `updateState()` reads the (optional) command messages, invokes the algorithm, and writes `thrForceCmdOutMsg`. Calling `updateState()` before `reset()` throws `XmeraLifecycleException`.

Algorithm layer (`forceTorqueThrForceMappingAlgorithm.h/.cpp`) The pure FP32 algorithm with no framework dependencies. The immutable `ForceTorqueThrForceMappingConfig` (created via the static `::create` factory) carries the thruster geometry, center of mass, and controllability assertions. Validators are: `numThrusters` $\in [1, \text{MAX_EFF_CNT}]$, each active direction $\hat{\mathbf{g}}_{t_i}$ within $1\text{e-}3$ of unit norm, and `centerOfMass_B` finite (`Eigen::Vector3f::allFinite()`). The static `::create` factory is the only place that throws `fsw::invalid_argument`: on an invalid thruster array or center of mass, on an asserted `desiredControlAxes_B` axis that is uncontrollable, or on an ill-conditioned thruster geometry (condition number above 100). The constructor and `setConfig` then cache the pseudo-inverse from the validated config; `update()` is `const` and never throws.

C shim (`forceTorqueThrForceMappingAlgorithm.c.h/.cpp`) Pure-C extern "C" wrapper around the algorithm for Ada (Adamant) FFI. The opaque `ForceTorqueThrForceMappingAlgorithm` handle is created from a `ForceTorqueThrForceMappingConfig_c` POD (`ThrusterArrayConfiguration_c` count + per-thruster `float[3]` arrays, `Vector3f_c` CoM, `uint8_t[6]` axes). The `configFromC` helper bridges the C/C++ representations via `architecture/utilities/eigenSupport.h`.

5 Module Parameters

Table 2: Module Parameters

Parameter	Type	Description
<code>desiredControlAxes_B</code>	<code>std::array<bool, 6></code>	Per-axis controllability assertion in body frame B . Entries <code>[0..2]</code> correspond to torque components τ_x, τ_y, τ_z and entries <code>[3..5]</code> correspond to force components F_x, F_y, F_z . A <code>true</code> entry asserts that the corresponding axis must be controllable by the configured thruster array; the assertion is cross-checked against the SVD of $[D]$ in <code>create()</code> (invoked during <code>reset()</code>). If any flagged axis lies outside the column space of $[D]$, the module throws <code>fsw::invalid_argument</code> (which surfaces in Python as <code>RuntimeError</code>). Default is all-true (full controllability asserted); set entries to <code>False</code> to opt out per axis when the thruster array is intentionally rank-deficient.

6 Initialization

The module is configured by:

```
module = forceTorqueThrForceMapping.ForceTorqueThrForceMapping()
module.modelTag = "forceTorqueThrForceMappingTag"
# Phase 1 (set before InitializeSimulation -> reset() builds the validated config):
# optional per-axis controllability assertion (torque xyz, force xyz in body frame B).
module.desiredControlAxes_B = [True, True, True, True, True, False]
```

The `cmdForceInMsg` and `cmdTorqueInMsg` are optional; if not connected the corresponding commanded vector is treated as zero. The `thrConfigInMsg` and `vehConfigInMsg` are required:

```

fswSetupThrusters.clearSetup()
for i in range(numThrusters):
    fswSetupThrusters.create(rcsLocationData[i], rcsDirectionData[i], maxThrust)
thrConfigInMsg = fswSetupThrusters.writeConfigMessage()

vehConfigInMsgData = messaging.VehicleConfigMsgPayload()
vehConfigInMsgData.CoM_B = CoM_B
vehConfigInMsg = messaging.VehicleConfigMsg().write(vehConfigInMsgData)

```

The relevant messages must then be subscribed to by the module:

```

module.cmdTorqueInMsg.subscribeTo(cmdTorqueInMsg)
module.cmdForceInMsg.subscribeTo(cmdForceInMsg)
module.thrConfigInMsg.subscribeTo(thrConfigInMsg)
module.vehConfigInMsg.subscribeTo(vehConfigInMsg)

```

7 Detailed Module Description

The following text describes the mathematics behind the `forceTorqueThrForceMapping` module.

7.1 Force and Torque Mapping

The desired force and torque are given as $\mathbf{F}_{req}$ and $\boldsymbol{\tau}_{req}$, respectively. These are both stacked into a single vector

$$\begin{bmatrix} \boldsymbol{\tau}_{req} \\ \mathbf{F}_{req} \end{bmatrix}$$

The i -th thruster position expressed in spacecraft body-fixed coordinates is given by $\mathbf{r}_i$. The unit direction vector of the thruster force is $\hat{\mathbf{g}}_{t_i}$. The thruster force is given as

$$\mathbf{F}_i = F_i \hat{\mathbf{g}}_{t_i}$$

The torque produced by each thruster about the body-fixed CoM is

$$\boldsymbol{\tau}_i = ((\mathbf{r}_i - \mathbf{r}_{COM}) \times \hat{\mathbf{g}}_{t_i}) F_i = \mathbf{d}_i F_i$$

The total force and torque on the spacecraft may be represented as

$$\begin{bmatrix} \boldsymbol{\tau}_{req} \\ \mathbf{F}_{req} \end{bmatrix} = \begin{bmatrix} \mathbf{d}_1 \cdots \mathbf{d}_N \\ \hat{\mathbf{g}}_{t_1} \cdots \hat{\mathbf{g}}_{t_N} \end{bmatrix} \begin{bmatrix} F_1 \\ \vdots \\ F_N \end{bmatrix} = [D] \mathbf{F}$$

The force required by each thruster is computed via the Moore-Penrose pseudo-inverse of $[D]$, formed from its truncated singular value decomposition $[D] = U \Sigma V^T$:

$$\mathbf{F} = V \tilde{\Sigma}^+ U^T \begin{bmatrix} \boldsymbol{\tau}_{req} \\ \mathbf{F}_{req} \end{bmatrix}$$

where $\tilde{\Sigma}^+$ is the diagonal of inverse singular values with entries below $\sigma_{\max} \cdot \varepsilon_{f32} \cdot \max(6, N_{\max})$ zeroed. This automatically drops any uncontrollable direction in the 6-D command space — equivalent to removing

rows of $[D]$ whose contribution lies entirely in the noise floor — without requiring an explicit row-by-row threshold.

The pseudo-inverse output $\mathbf{F}_{pre} = [D]^+ \text{cmd}$ may contain negative entries when the commanded force/torque pushes some thrusters into a sign opposite their direction vector. To ensure no commanded thrust is less than zero, the minimum thrust is subtracted from the thrust vector:

$$\mathbf{F} = \mathbf{F}_{pre} - \min(\mathbf{F}_{pre}) \cdot \mathbf{1}$$

7.2 Balanced Layouts and Null-Space Optimality

The shift in `eq:F_min` is FT-preserving precisely when the thruster layout is **balanced**, i.e. when $[D]\mathbf{1} = 0$. Expanding the rows of $[D]$, balance is equivalent to two geometric conditions on the thruster array:

$$\sum_i \hat{\mathbf{g}}_{t_i} = \mathbf{0}, \quad \sum_i (\mathbf{r}_i - \mathbf{r}_{\text{COM}}) \times \hat{\mathbf{g}}_{t_i} = \mathbf{0}$$

When both hold, $\mathbf{1} \in \ker([D])$ and shifting along $\mathbf{1}$ stays inside the affine set $\{\mathbf{F} : [D]\mathbf{F} = \text{cmd}\}$, leaving the achieved force/torque exactly equal to the commanded value (modulo SVD truncation of uncontrollable directions).

A further useful property of the pseudo-inverse on a balanced layout: $\mathbf{F}_{pre}$ lies in the row space of $[D]$, which is orthogonal to $\ker([D])$. Since $\mathbf{1} \in \ker([D])$, this implies $\mathbf{1}^T \mathbf{F}_{pre} = 0$ — the raw pseudo-inverse output has positive and negative entries that exactly cancel — and the post-shift total thrust is therefore

$$\sum_i F_i = -N \cdot \min(\mathbf{F}_{pre})$$

where N is the number of thrusters.

On a balanced layout with zero commanded body force, the single-direction shift along $\mathbf{1}$ is globally fuel-optimal — no more general null-space optimization can reduce the total thrust further. Verified empirically on the 8-thruster reference layout across 15 representative torque commands (including 6 random vectors): the min-shift solution and the full linear program $\min \mathbf{1}^T \mathbf{F}$ subject to $[D]\mathbf{F} = \text{cmd}$ and $\mathbf{F} \geq 0$ produce identical total thrust to within fp32 noise. The layout's bilateral symmetry constrains the pinv image so that exploring the 1-D shift along $\mathbf{1}$ reaches the same linear programming (LP) vertex value that the full $\dim \ker([D])$ -dimensional null space would. Layouts with lower symmetry or nonzero commanded body force can break this equivalence and create headroom for a more general null-space optimization (e.g. an LP over $\ker([D])$) — but the current configuration does not benefit, so the module retains the simpler single-direction min-shift.

On **unbalanced** layouts ($[D]\mathbf{1} \neq 0$), the min-shift perturbs the achieved force/torque by $-\min(\mathbf{F}_{pre}) \cdot ([D]\mathbf{1})$.

8 Additional Information

For rank-deficient $[D]$ the minimum-norm solution is returned; uncontrollable directions in the 6-D command space are silently dropped by the SVD truncation. The optional `desiredControlAxes.B` parameter converts that silent drop into a fail-fast assertion at configuration time: if any axis flagged `True` in `desiredControlAxes.B` lies outside the column space of $[D]$, `create()` (invoked from the adapter's `reset()`) throws `fsw::invalid_argument` rather than returning a solution that ignores the request. `create()` likewise rejects an ill-conditioned thruster geometry (kept condition number above 100), where the pseudo-inverse would amplify the command and fp32 round-off.